

Отчёт по проекту

Студент: Васильева Софья Александровна **Группа:** БИВ233

1. Введение и постановка задачи

- **Цель проекта:**

Предсказать, будет ли авиапассажир удовлетворён перелётом, по характеристикам поездки, класса обслуживания, задержек и оценок качества сервиса.

Такая задача может быть полезна авиакомпании: модель помогает понять, какие факторы сильнее связаны с удовлетворённостью пассажиров, и заранее находить случаи с высоким риском неудовлетворённости.

- **Формулировка задачи:**

Решается задача бинарной классификации.

Целевой признак: `satisfaction`.

Классы:

- `satisfied` — пассажир удовлетворён;
- `dissatisfied` — пассажир не удовлетворён.

Положительный класс: `satisfied`.

- **Обоснование метрики качества:**

Основная метрика — ROC-AUC.

ROC-AUC выбрана как основная метрика, потому что модель возвращает вероятность принадлежности пассажира к классу `satisfied`, а нам важно сравнивать модели по качеству ранжирования объектов. Чем выше ROC-AUC, тем лучше модель в среднем ставит более высокие вероятности удовлетворённым пассажирам.

Дополнительные метрики: Accuracy, Precision, Recall, F1-score. Они используются для интерпретации итоговых бинарных предсказаний, но при выборе модели приоритет отдаётся ROC-AUC.

2. Поиск и описание данных

- **Источник данных:**

Датасет Airline Customer Satisfaction Prediction с Kaggle:

<https://www.kaggle.com/datasets/raminhuseyn/airline-customer-satisfaction>

- **Описание датасета:**

- количество строк: 129 880;
- количество исходных колонок: 22;
- целевой признак: `satisfaction`;
- задача: бинарная классификация.

Группы признаков:

- признаки пассажира: `customer_type`, `age`;
- признаки поездки: `type_of_travel`, `class`, `flight_distance`;
- оценки сервиса: `seat_comfort`, `food_and_drink`, `inflight_wifi_service`, `online_support`, `online_boarding`, `cleanliness` и другие;
- задержки: `departure_delay_in_minutes`, `arrival_delay_in_minutes`.

Данных достаточно: датасет содержит больше 10 000 строк и больше 10 колонок.

3. Обработка и подготовка данных

Обработка данных выполнена в ноутбуке `notebooks/01_EDA.ipynb`.

- **Полная очистка:**

- Пропуски: числовые признаки заполнялись медианой, категориальные — самым частым значением. Для финальных выборок статистики считались на train-части.
- Дубликаты: полные дубликаты были удалены.
- Выбросы: для признаков задержек использовался clipping по 99-му перцентилю. Порог рассчитывался на train-части.
- Типы данных: проверка и приведение к корректным типам.

- **Работа с фичами:**

Добавлены новые признаки:

- `total_delay` — суммарная задержка вылета и прилёта;
- `has_delay` — был ли рейс задержан;
- `is_long_flight` — длинный ли перелёт;
- `mean_service_score` — средняя оценка сервиса.

Эти признаки агрегируют информацию о задержках и качестве обслуживания.

- **Визуализации:**

В EDA были построены графики:

- распределение `target`;
- пропуски;
- boxplot задержек;
- satisfaction rate по `class`;
- satisfaction rate по `type_of_travel`;

- satisfaction rate по `customer_type`;
- распределение `mean_service_score`;
- correlation heatmap.

По графикам видно, что удовлетворённость связана с классом обслуживания, типом поездки и оценками сервиса. Особенно заметно влияние бизнес-класса и сервисных оценок.

- **Сплит данных:**

Данные разделены на train / validation / test в пропорции 70% / 15% / 15%.

Использовалась стратификация по target, чтобы сохранить баланс классов.

Как избегали data leakage:

- `satisfaction` не используется как feature;
- вспомогательная колонка `target` не используется как feature;
- статистики preprocessing считаются на train-части;
- validation и test не используются при обучении моделей;
- преобразования признаков выполняются внутри `sklearn Pipeline`.

4. Baseline-модель

Baseline-модели обучались в ноутбуке `notebooks/02_Baseline.ipynb`.

- **Модель:**

В качестве нижней границы качества использовался `DummyClassifier`, который предсказывает самый частый класс.

Также использовалась простая baseline-модель `LogisticRegression`.

- **Результаты:**

Модель	ROC-AUC	Комментарий
DummyClassifier	0.500000	Нижняя граница качества
LogisticRegression	~0.902	Простая baseline-модель

- **Цель:**

Baseline нужен как точка отсчёта. Более сложные модели должны показывать качество выше dummy-модели и простой логистической регрессии.

5. Эксперименты

Эксперименты проводились в ноутбуках:

- `notebooks/02_Baseline.ipynb`;
- `notebooks/03_Experiments.ipynb`.

Модель / эксперимент	Гипотеза	Как проверялось	Результат
DummyClassifier	Нужно получить нижнюю границу качества	Модель предсказывает самый частый класс	ROC-AUC = 0.500000
LogisticRegression	Простая линейная модель может дать разумный baseline	Обучение на train, проверка на validation	ROC-AUC около 0.902
DecisionTree	Дерево может учитывать нелинейные зависимости	Проверены разные значения <code>max_depth</code>	Качество выше baseline, но хуже ансамблей
RandomForest	Ансамбль деревьев должен улучшить качество	Проверены разные <code>max_depth</code> , затем <code>RandomizedSearchCV</code>	Validation ROC-AUC = 0.991223 после тюнинга
GradientBoosting	Последовательное построение деревьев может лучше улавливать зависимости	<code>RandomizedSearchCV</code> по <code>n_estimators</code> , <code>learning_rate</code> , <code>max_depth</code>	Validation ROC-AUC = 0.991339
Feature importance	Нужно понять, какие признаки важны для модели	Извлечена важность признаков у финальной модели	Самые важные: <code>inflight_entertainment</code> , <code>seat_comfort</code> , <code>ease_of_online_booking</code>

- **Перебор гиперпараметров:**

Для перебора использовался `RandomizedSearchCV`.

Для `RandomForest` подбирались:

- `n_estimators`;
- `max_depth`;
- `min_samples_split`;
- `min_samples_leaf`;
- `max_features`.

Лучшие параметры `RandomForest`:

```
n_estimators = 279
max_depth = 16
max_features = None
min_samples_leaf = 1
min_samples_split = 7
```

Для GradientBoosting подбирались:

- `n_estimators`;
- `learning_rate`;
- `max_depth`;
- `min_samples_split`;
- `min_samples_leaf`.

Лучшие параметры GradientBoosting:

```
n_estimators = 250
learning_rate = 0.07
max_depth = 5
min_samples_leaf = 1
min_samples_split = 4
```

Таблица tuned-моделей:

Модель	Dataset	ROC-AUC	F1-score	Accuracy	Precision	Recall
gradient_boosting_tuned	validation	0.991339	0.954270	0.950416	0.963483	0.945231
random_forest_tuned	validation	0.991223	0.954513	0.950672	0.963589	0.945606
gradient_boosting_tuned	test	0.991728	0.954134	0.950108	0.960205	0.948138

GradientBoosting выбран как финальная модель, потому что он получил лучший ROC-AUC на validation.

6. Финальная модель и интерпретируемость

Финальная модель:

Финальная модель — `GradientBoostingClassifier` с параметрами:

```
learning_rate = 0.07
max_depth = 5
min_samples_leaf = 1
min_samples_split = 4
n_estimators = 250
```

Для деплоя модель была переобучена на train + validation и проверена на test.

Финальное качество на test:

Метрика	Значение
Accuracy	0.951186
Precision	0.961422

Метрика	Значение
Recall	0.948889
F1-score	0.955114
ROC-AUC	0.991833

Обоснование выбора:

GradientBoosting выбран, потому что он показал лучший ROC-AUC на validation. При этом test-качество близко к validation-качеству, значит сильного переобучения не видно.

Интерпретируемость:

Для финальной модели был построен feature importance.

Самые важные признаки:

1. `inflight_entertainment`;
2. `seat_comfort`;
3. `ease_of_online_booking`;
4. `class_Business`;
5. `departure_arrival_time_convenient`.

Это показывает, что удовлетворённость пассажиров сильнее всего связана с качеством сервиса, комфортом и классом обслуживания.

7. Деплой

Для CP3 был реализован локальный деплой модели в двух вариантах:

- FastAPI API для отправки запросов к модели;
- Streamlit-интерфейс для ручного ввода признаков и получения предсказания.

Интерфейс

Интерфейс реализован с помощью Streamlit в файле:

```
app/streamlit_app.py
```

В интерфейсе пользователь может задать параметры пассажира и перелёта:

- тип клиента;
- возраст;
- тип поездки;
- класс обслуживания;
- дистанцию перелёта;
- оценки качества сервиса;
- задержку вылета и прилёта.

После нажатия кнопки приложение выводит:

- вероятность класса `satisfied`;
- итоговый предсказанный класс: `satisfied` или `dissatisfied`.

Скриншоты Streamlit-интерфейса сохранены отдельно, потому что форма содержит много полей и не помещается на один экран:

- `streamlit_form.png` — пример формы для ввода параметров пассажира;
- `streamlit_prediction.png` — результат предсказания после нажатия кнопки.

API

API реализован с помощью FastAPI в файле:

```
api/app.py
```

Реализованы два endpoint:

Endpoint	Метод	Назначение
<code>/health</code>	GET	Проверяет, что сервис работает и модель доступна
<code>/predict</code>	POST	Принимает признаки пассажира и возвращает предсказание

Endpoint `/predict` принимает JSON с признаками пассажира, параметрами поездки, оценками сервиса и задержками. Внутри API выполняется тот же feature engineering, что и при обучении модели: рассчитываются `total_delay`, `has_delay`, `is_long_flight` и `mean_service_score`.

Пример ответа API:

```
{
  "prediction": 1,
  "prediction_label": "satisfied",
  "satisfied_probability": 0.949490076459031
}
```

Скриншоты работы API сохранены в:

```
report/images/api_docs.png
report/images/api_predict.png
```

Скриншоты

В отчёт добавлены скриншоты:

- `api_docs.png` — документация FastAPI;

- `api_predict.png` — пример запроса к `/predict`;
- `streamlit_form.png` — форма для ввода параметров пассажира;
- `streamlit_prediction.png` — пример работы Streamlit-интерфейса.

FastAPI docs

Airline Satisfaction Prediction API

OAS 3.1

[/openapi.json](#)

default

GET	/health	Health	⌵
POST	/predict	Predict	⌵

Schemas

HTTPValidationError	>	Expand all	object
PassengerInput	>	Expand all	object
ValidationError	>	Expand all	object

FastAPI predict

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "customer_type": "Loyal Customer",
    "age": 35,
    "type_of_travel": "Business travel",
    "travel_class": "Business",
    "flight_distance": 1000,
    "seat_comfort": 4,
    "departure_arrival_time_convenient": 5,
    "food_and_drink": 4,
    "gate_location": 3,
    "inflight_wifi_service": 4,
    "inflight_entertainment": 4,
    "online_support": 4,
    "ease_of_online_booking": 4,
    "on_board_service": 4,
    "leg_room_service": 4,
    "baggage_handling": 4,
    "checkin_service": 4,
    "cleanliness": 5,
    "online_boarding": 4,
    "departure_delay_in_minutes": 0,
    "arrival_delay_in_minutes": 0
  }'
```

Request URL

http://127.0.0.1:8000/predict

Server response

Code

Details

200

Response body

```
{
  "prediction": 1,
  "prediction_label": "satisfied",
  "satisfied_probability": 0.949490076459031
}
```



Download

Response headers

```
content-length: 89
content-type: application/json
date: Tue, 26 May 2026 18:30:28 GMT
server: uvicorn
```

Responses

Streamlit form

Предсказание удовлетворённости авиапассажира

Тип клиента

Loyal Customer

Возраст

35

Тип поездки

Business travel

Класс обслуживания

Business

Дистанция перелёта

1000

Оценки сервиса

Seat comfort

4

Departure/Arrival time convenient

4

Food and drink

4

Gate location

3

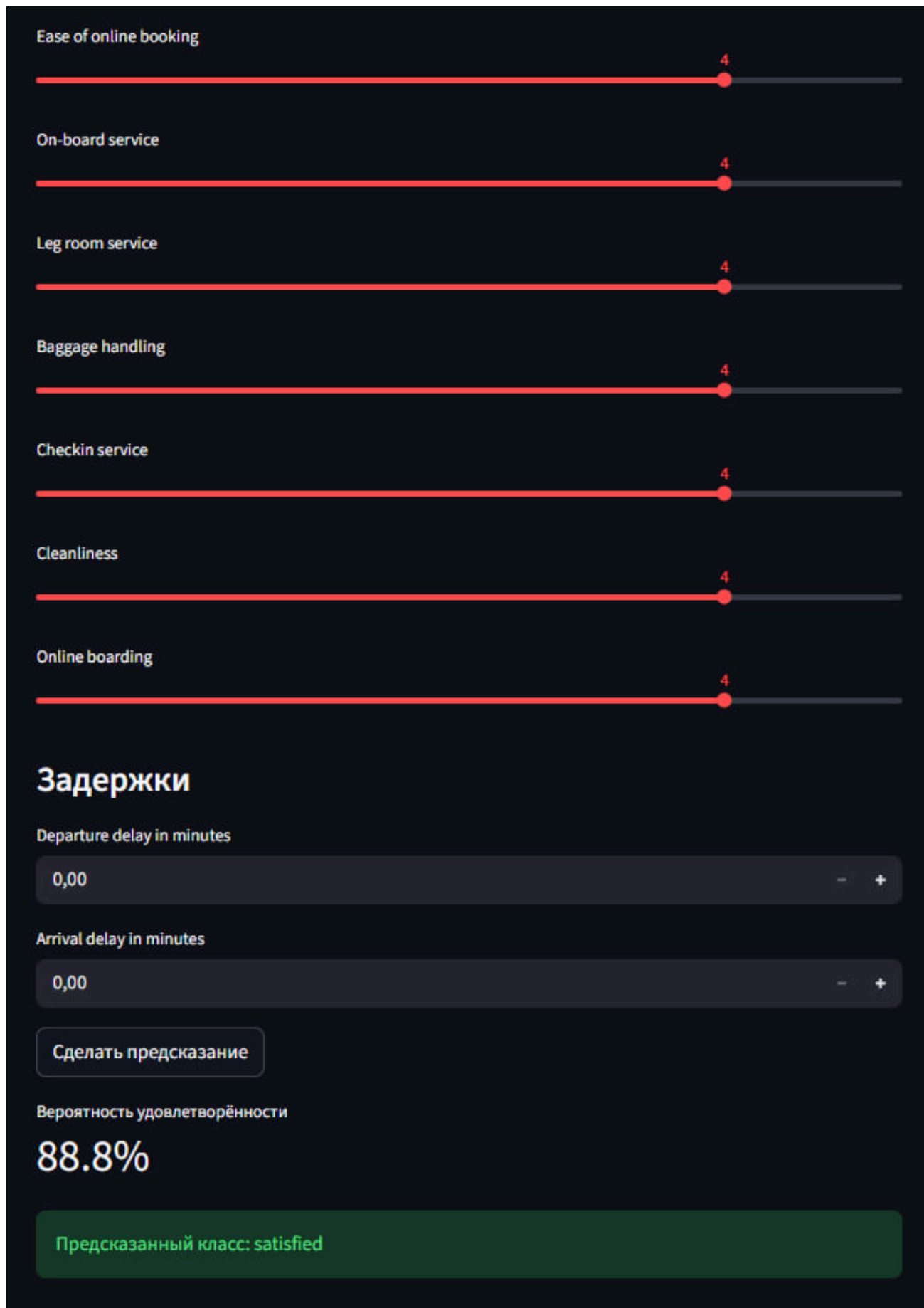
Inflight wifi service

4

Inflight entertainment

4

Streamlit prediction



Видео

Ссылка на видео демонстрации работы:

<https://disk.yandex.ru/i/R6DeGPolCgNDLw>

8. Заключение и выводы

- **Итоги:**

В проекте была решена задача бинарной классификации удовлетворённости авиапассажира.

В ходе работы:

- данные были загружены и проанализированы;
- выполнена очистка данных;
- обработаны пропуски, дубликаты и выбросы;
- добавлены новые признаки;
- выполнен train / validation / test split без утечки целевой переменной;
- обучены baseline-модели;
- проведены эксперименты с несколькими ML-моделями;
- выполнен подбор гиперпараметров через `RandomizedSearchCV`;
- выбрана финальная модель;
- проведён анализ важности признаков;
- реализованы FastAPI и Streamlit для локального деплоя.

Финальная модель — `GradientBoostingClassifier`.

Финальное качество на test-выборке после обучения на train + validation:

Метрика	Значение
ROC-AUC	0.991833
F1-score	0.955114
Accuracy	0.951186
Precision	0.961422
Recall	0.948889

По сравнению с baseline качество значительно выросло:

Модель	ROC-AUC
DummyClassifier	0.500000
LogisticRegression	~0.902
Финальная GradientBoosting	0.991833

- **Ограничения:**

У проекта есть несколько ограничений:

- модель обучалась на готовом Kaggle-датасете, а не на актуальных данных конкретной авиакомпании;
- threshold для перевода вероятности в класс отдельно не подбирался;
- деплой реализован локально;
- **Возможные улучшения:**

В дальнейшем можно улучшить проект следующим образом:

- подобрать оптимальный threshold под конкретную бизнес-задачу;
- протестировать дополнительные бустинговые модели, например CatBoost или LightGBM;
- добавить сохранение истории запросов к API;
- развернуть сервис на сервере.